Rochester Institute Of Technology

December 15, 2013

Best Time To Vacation

Wander Bravo, Piter Garcia, Anshika Garg

ABSTRACT

The aim of this project is to learn some Big Data and Data Mining techniques and implement them in a project chosen by our team. The name of our project is "*Best Time To Vacation*", and we will be using weather datasets of the United States to analyze, compare, and show the weather trends in particular areas. Thus, we will be able to approximate the weather and precipitation of those places requested by the user on a designated time range. Moreover, we will be able to suggest places that most users may like. As a result, we have designed an interface that will allow users to monitor the weather of a city, including its precipitation, for short-term vacationing purpose.

1. OVERVIEW

The logical structure of our project is composed of three main phases, which better describe the milestones achieved throughout our project development. These phases are classified as Design, Architecture and Implementation. Each phase explains in detail the main steps taken to model and structure a database with our interface to meet the project specifications. In order to better organize our documentation we have divided these phases into two main sections, which contain a summary of the work process done in our project. These two sections are Database and Website, and their development can be described as follows:

In the database section three main divisions take place in order to make the database usable. These divisions include Formatting, Design, and Tables. First, in formatting, we will elaborate how our datasets are formatted, the process taken to make our datasets easy to read by avoiding noisy data and giving them an appropriate format. Second, in Design, we will explain the design of our database in terms of appearance, structure and performance. Lastly, in Tables, we will describe the process taken to load and distribute our datasets into their appropriate tables. Also, we will explain how each table is generated as well as their functionality.

The Website development process is explained within the following three divisions; design, registration, and features. First, in the Design, our interface is designed according to the appearance, structure, and performance we wanted our project to achieve. Second, in registration, we will describe the process taken to create the appearance, structure, as well as the techniques used to allow users to access our system in an efficient manner. Third, in Features, we describe the functionality for each of the features to be performed through the interface via our database and system analysis.

In particular, we will explain some tools and techniques used to achieve our goals in each one of the divisions explained in the Database and Website sections. These divisions are explained separately in each phase as the process taken in each one varied.

2. DESIGN

In this project, the operational process of our design meets the objective of a high quality development which responds to real life scenarios in big data. The focus of this step is the development of an acceptable functional solution to the requirements of our project with an exterior design solution that is compatible with our database, interface, and our goal as a whole. The progress sets of our design were formally modified, implemented, and reviewed by our group as they were issued at the completion of our schematic design, design development, and during the development of our database. A successful design development was needed in order to coordinate the activities of users and project architecture to meet our goal, as these are important for our overall project development. In fact, our project design process typically involves the sequence of activities described in the following divisions.

2.1 Database

The datasets consist of the temperature maximum and minimum, precipitation, wind speed, longitude and latitude (position about 1/8 degrees apart) of some locations. These datasets have been collecting weather information for the past forty four years, which has helped us analyze the trends of some areas over the past years and made our results more reliable. The main goal was to design a database, from which we can easily generate tables for the cities, cities' s weather information, and user' s cities selections as shown in the image in the figure 1.

Tables	Attributes						
Person	name	password				minTemperature	maxTemperature
Search	area	startDate	endDate				
Precipitation	year	month	date	longitude	latitude	minPrecipitation	maxPrecipitation
Temperature	year	month	date	longitude	latitude	min temperature	maxTemperature
Place	area			longitude	latitude)		

Figure 1: Tables Design.

1. **Formatting**
   The main goal was to make querying for the weather information easy and reliable. In order to make a better use of the database, we have formatted each dataset to remove redundant, noisy, and unnecessary data, which helped our system to run more efficiently.

   - **Appearance**
     The main purpose for the formatting process was to make our database look as the images below. In this way, we are able to read the formatted data files according to the requirements of our project. This process allows us to easily relate each one of the input dates to the corresponding city and its temperature, within the given range, to approximate the city' s weather condition within the given dates.

   - **Structure**
     In order to create the desired appearance described above, we needed to make our database go through some steps to make this structure possible. First, the datasets were compressed and therefore we needed to extract them by using the little Edian format, which was a very difficult process to include in our code. Second, we sorted the cities and separated them from those we did not plan to include in our database. Also, the datasets were huge and unnecessary for our implementation. Therefore, we modified the datasets to contain only weather information from 1980 to 2013. This, according to obtained data, includes enough data for the purpose of our project. Lastly, each dataset was formatted to relate to the weather, user, and longitude-and-latitude tables to its corresponding city by means of an id, which enabled us to match a city weather records' dates to the user given dates.

   - **Performance**
     The datasets we obtained were huge and very difficult to handle. The formatting process was a very important step in our project to improve and speed up every process. At first, it was very difficult for us to test our features because the system was slow and overloaded with data. This is why we proceeded to exclude a large amount of cities and only work with some cities to make testing more efficiency and reliable.

2. **Design**
   We designed the appearance and structure of our database to perform according to the datasets obtained as mentioned above.

   - **Appearance**
     The purpose of our design was to allow the tables in our database to relate to the input dates, corresponding city and its temperature. Thus, we would be able to approximate the city' s weather condition within the given range of the input dates.

   - **Structure**
     The main structure of our database was based on the Temperature, City, Longitude-and-Latitude Tables. The structure of these were developed by tools that are explained in the architecture phase of this project. An example of the design of these tables' structure can be seen in the figures 4 and 5.

     User's table had a different structure to this since this table is populated as users register. The structure of this was developed by tools that are explained in the architecture section of this project. The structure design of this table can be seen in the figure 7.

   - **Performance**
     The performance of each table in our database is related to the user's table. After the user has signed into our system, this will populate the user's table accordingly. The content of this will access to the other tables such as the city, longitude-and-latitude, and weather tables. Once it has gathered the necessary data it will proceed to do some data processing techniques, explained in the implementation phase, to approximate the weather condition of a city, selected by the user, within the dates. This performance is explained in detail in the architecture phase of this project.

```
--
-- Dumping data for table `city`
--

INSERT INTO `city` (`id`, `name`, `latitude`, `longitude`) VALUES
(1, 'Montgomery', 32.23, -86.22),
(2, 'Bridgeport', 41.11, -73.11),
(3, 'Washington', 38.51, -77.2),
(4, 'Miami', 25.48, -80.16),
(5, 'Orlando', 28.33, -81.23),
(6, 'Dublin', 32.2, -82.54),
(7, 'Atlanta', 33.39, -84),
(8, 'Baltimore', 39.11, -76.4),
(9, 'Boston', 42.22, -71.2),
(10, 'Trenton', 40.13, -74.46),
(11, 'Albany', 42.45, -73.48),
(12, 'Raleigh', 35.52, -78.47),
(13, 'NYC', 40.47, -73.58);
```

Figure 2: Table to Insert Cities.

```
INSERT INTO `data` (`id`, `year`, `month`, `day`, `prec`, `tmax`, `tmin`, `wind`) VALUES
(2, 1980, 1, 1, 1, 0, 7, 0),
(2, 1980, 1, 2, 25, 10, 1, 1),
(2, 1980, 1, 3, 1, 0, 9, 0),
(2, 1980, 1, 4, 26, 10, 3, 3),
(2, 1980, 1, 5, 1634, -1, 3, 0),
(2, 1980, 1, 6, 10, 7, -1, 8),
(2, 1980, 1, 7, 0, 0, 10, 1),
(2, 1980, 1, 8, 20, 8, 0, 5),
(2, 1980, 1, 9, 1633, -1, 1, 0),
(2, 1980, 1, 10, 11, 4, -3, 14),
(2, 1980, 1, 11, 0, 0, 10, -1),
(2, 1980, 1, 12, 31, 12, 0, 11),
(2, 1980, 1, 13, 0, 0, 8, 0),
(2, 1980, 1, 14, 29, 11, 1, 3),
(2, 1980, 1, 15, 1, 0, 10, 1),
(2, 1980, 1, 16, 6, 2, 11, 8),
(2, 1980, 1, 17, 7, 3, 7, 2),
```

Figure 3: Table to Insert Data(weather).

3. **Tables**
   The main tables used to create our database are the Data and City tables. These tables were created, after uncompressing the data, with an algorithm we coded in java. This process is explained in detail in the architecture phase; we will briefly explain how this process took place.

- **Appearance**

```
-- Table structure for table `city`
--

CREATE TABLE `city` (
  `id` int(11) NOT NULL,
  `name` varchar(25) NOT NULL,
  `latitude` float NOT NULL,
  `longitude` float NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

Figure 4: City Table.

In order to load our tables into our system it was necessary for us to have the two tables, mentioned above, formatted.

```
--
-- Table structure for table `data`
--

CREATE TABLE `data` (
  `id` int(11) NOT NULL,
  `year` int(11) NOT NULL,
  `month` int(11) NOT NULL,
  `day` int(11) NOT NULL,
  `prec` int(11) NOT NULL,
  `tmax` int(11) NOT NULL,
  `tmin` int(11) NOT NULL,
  `wind` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

Figure 5: Data(Weather) Table.

- **Structure**
  The structure of each table should be as follows. First, the City table, this is composed by latitude, longitude, and the cityName. Lastly, the Data table, which is composed by the date, precipitation, temperature, and wind of each city. This is the structure we used to design our database as shown in the figures 4 and 5.

- **Performance**
  The City and Data tables are used by the user' s table to implement the features of our system. This includes weather approximation, as well as to suggest cities that the user may like according to the user' s behavior.

2.2 Interface

We have designed a website that works as a platform to shape and implement our project according to our expectations. In order to fulfill the design and project patterns, we are going to break down our approach into the following divisions.

1. **Design**
   The way we have designed our website is according to the database design. We have created a set of features that users will be able to use in our website through our database implantation. In order for users to be able to access these features they first must sign in into our system. This is why we have described our interface section in the following divisions.

- **Appearance**

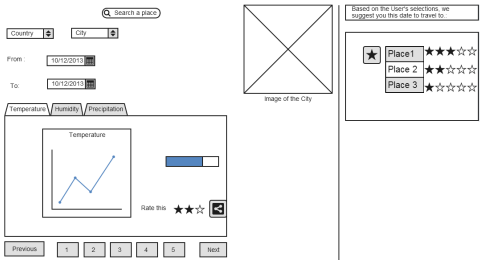

Figure 6: Search Interface Design

An example of the appearance of our interface, or website, design is in the figure 8. The appearance of our website is highly influenced by the architecture of our project. Therefore, even though the design of our website appearance may not correlate with the implementation, the features are the same.

- **Structure**

```
CREATE TABLE "searchPlace_userprofile" (
    "id" serial NOT NULL PRIMARY KEY,
    "about_me" text,
    "facebook_id" bigint UNIQUE,
    "access_token" text,
    "facebook_name" varchar(255),
    "facebook_profile_url" text,
    "website_url" text,
    "blog_url" text,
    "date_of_birth" date,
    "gender" varchar(1),
    "raw_data" text,
    "facebook_open_graph" boolean,
    "new_token_required" boolean NOT NULL,
    "image" varchar(255),
    "user_id" integer NOT NULL UNIQUE REFERENCES "django_facebook_facebookcustomuser" ("id")
DEFERRABLE INITIALLY DEFERRED
);
```

Figure 7: Search-Place Structure Design

We used certain tools in order to provide our interface with the structure needed to meet our project specifications. For this project, PostgreSQL is the database engine underlined by our management system to create, read, update and delete data from the database. Moreover, we have included other database and web development software that have helped us to model and process our database faster and easier. The application-programming interface that we are using, to allow users to interact with our underlying engine without going through the user interface of our system, is Django. We are also making use of tools such as Python and HTML. These tools are described in detail in the architecture section of our project, and examples of these are given to show the reader how the structure of our interface took place.

- **Performance**
  The platform that we are using to manage our database is Django. The installation and upgrading process is very easy since we have the ability to automatically install database adapters and other dependencies. Using Django in our management systems has allowed us to extract valued content from our datasets and build a high performing web applications system in an elegant and efficient way. The following factors have been improved with the use of Django. First, we can define our data models much easier with Python which provides us with a dynamic database access APIA that has been formatted with SQL. Second, it provided us with an automatic admin interface that adds and updates users and user's content automatically. Third, it supports Multilanguage applications. This is essential for our implementation since it is used mostly for vacation purposes. Finally, we can always use cache frameworks to increase future performance.

2. **Features**
   All features of our project were designed to meet our main goal. This is to help users to decide which places to visit while on vacation. In order to achieve this, we need to provide users with features that allow them to search for a place, monitor the approximate weather information, and have a list of suggested cities that users may want to visit according to their behavior. The design of these features can be described as follows:

Figure 8: Sign-In Interface Design

- **Registration**
  The user needs to log in by providing a username and a password. This will assign each user with a unique id and password. An example of this feature can be seen in the figure 8. We have an application for users to log into our system through their Facebook accounts. This feature facilitates users to utilize our system in a more efficient manner, and improves our system as follows. First, it helps us to increase user registrations quickly. Second, it provides us with an app id and app secret-id; this way the user does not have to worry about creating a new username and password. Overall, this feature could potentially help us integrate more features and allow for future improvements.

- **Search**

```
CREATE TABLE "searchPlace_place" (
    "id" serial NOT NULL PRIMARY KEY,
    "city" varchar(2) NOT NULL,
    "slug" varchar(50) NOT NULL UNIQUE,
    "longitude" numeric(5, 2) NOT NULL,
    "latitude" numeric(5, 2) NOT NULL
);
```

Figure 9: Sign-Place Table Design

  This is a very important feature because it will allow users to search and find cities of their interest. This feature will automatically add the searches done by the user; the list of searches or table will provide suggestions to the user with a list of cities that he or she may like. More detail of the structure of this feature is covered in the architecture phase of our project. The image in figure is an example of this feature's design.

- **Graph**
  In the graph feature, there is a function that we use to approximate the weather information of a city. Once the weather data was approximated, this data is accessed and displayed by the graph feature. The process in which we make use of the weather data to make the approximation is better explained in the architecture phase of our project.

- **Suggestion List**
  This feature shows a graphical representation of the approximated weather and precipitation of the designated city that the user has searched for. In this feature, we use the table that stores all the searches done by the user. We use the data in this table such as userâĂŹs information and the weather information of the cities. After all this information is gathered, we create another table that will contain the suggested cities that the user may want to visit. The structure of this features is explained in the architecture phase of our project.

3. ARCHITECTURE

It was critical to model the right architecture in order to succeed in our system design. The architecture system representation was essential for the description and analysis of the high level properties of our system. This essential tenet forced us to consider alternatives due to the architecture of our project. The architecture of our project specifies the overall structure of our system. This includes the communication, synchronization, and data access, which are described below in the database and interface sections. The database contains the assignment of functionality for our designed features, data corporal distribution, composition of designed features, scaling and performance, and data design modifications. In contrast, the interface section includes the

body of work for our module interconnection features, templates and frameworks for our system that serve the needs of our database domains, and formal models of component integration mechanisms. The two following sections will illustrate the architecture process done throughout our project development.

3.1 Database

1. Formatting

In the formatting process of our project we used some algorithm techniques to model our data to meet the project requirements. These algorithms along with some software tools that we used are being explain below.

- **Appearance**

  The appearance of the database was obtained through algorithms such as the little Edian. Also, we used our own Java code implementation to reduce the datasets to only 20 years of weather information, and to reduce the number of cities.

```
unsigned short int *prec;
short int *array1, *array2, *array3;

for (x=0;x<lines/4;x++) {
if(fread(&prec[x],sizeof(unsigned short int),1,fp) != 1)
   printf("read prec[x] err rec %d %s\n", x,file);
if(fread(&array1[x],sizeof(short int),1,fp) != 1)
  printf("read array1[x] err rec %d %s\n", x,file);
if(fread(&array2[x],sizeof(short int),1,fp) != 1)
  printf("read array2[x] err rec %d %s\n", x,file);
if(fread(&array3[x],sizeof(short int),1,fp) != 1)
  printf("read array3[x] err rec %d %s\n", x,file);

prcp = (float) (prec[x])/40.0;
tmax = (float) (array1[x])/100.0;
tmin = (float) (array2[x])/100.0;
wind = (float) (array3[x])/100.0;
}
```

Figure 10: Little Edian Algorithm

Little Edian is a compression format algorithm that we used to uncompress the datasets mentioned in the design section of our project. We had access to the algorithm through Washington University Website referenced below, and a demonstration of the algorithm is shown in figure 10.

We implemented in java the code shown in figure 11. This code reduces the data to 20 years of weather information, and it only loads the datasets from the west cost cities.

- **Structure**

  In order to use our database, we had to model each dataset to load and query easily. To do this, we have used our own code implementation that creates the city and data table explained in the designed section. These tables are created by the algorithm shown in the figure 11. This algorithm

```
while ((line=br.readLine())!=null)
{
     String [] parts = line.split("\t");
     int len = parts.length;
     fileOut.print (id+"\t"+month+"/"+date+"/"+year+"/t");
     if(date == day[month-1])
     {
          month++;
          date = 1;
          if(month > 12)
          {
               year++;
               month = 1;
          }
     }
     if(month == 2 && year%4 == 0)
     {
          day[month-1] = 29;
     }
     else
     {
          day[1] = 28;
     }
     for(int i = 0; i < len-1; i++)
     {
          fileOut.print (parts[i]+"\t");
     }
     fileOut.print (parts[len-1]+"\n");
}
```

Figure 11: Java Source Code - Data(Weather) Reduction

consist of two parts; the first is creating the city table and the second is creating the data table. An image of these two is shown in the figures 4 and 5.

- **Performance**

  The reduction and formatting of the datasets made easier the querying process. The algorithm mentioned in the structure of this division was necessary in order to perform our testing procedures. The reason for this is because the datasets were huge, and we did not need more than 20 years of weather data to approximate the weather of a city.

2. Design

- **Appearance and Structure**

  The structure of our database was modeled through the pgAdminIII work platform. This work platform helped us to generate the tables that we needed to make possible the functionalities of each features. Through this application we were able to generate the city and data tables, which are very essential for our implementation. A demonstration of the work platform is shown in the figure 12.

- **Performance**

  The structure of the pgAdmingIII platform devises a query plan for each query that is given to generate the tables. The generation of these tables will decrees the querying time, and gives us a close approximation in a quick step.

3. Tables

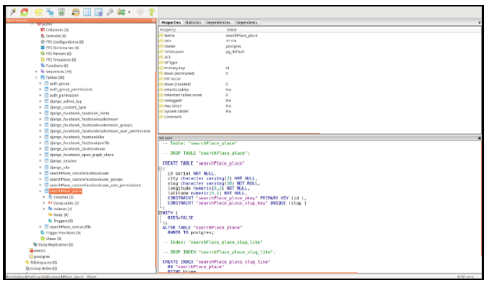

Figure 12: Work Platform to Create Tables

- **Data**

  The data table consist of weather information of the cities, these are temperature maximum and minimum, precipitation and wind speed. These weather information corresponds to each city id, and the data is from 1980 to 2013. The schema of this table is shown in the image above.

- **City**

  The data table for the city contains only the id, the longitude, and latitude of each city. The id is used reference to the Data table. The schema of this is shown in the figure 14.

- **Search**

  In the search place table, we have assigned to each area, which we named city in our query, a unique id through which we can distinguish it from the other areas. Each city has its own longitude and latitude and these values are provided by our database. The slug is required by our system, this is a way of generating a valid URL using the data already obtained, an example of this table is shown in the figure 15.

- **User**

  The user table is generated through Django by importing the data from Facebook to the user table. This table can later be used in the pgAdminIII work platform to query accordingly.

- **City data**

```
TABLES:
CREATE TABLE city_data
(
  id serial NOT NULL,
  city_id integer NOT NULL,
  year integer NOT NULL,
  month integer NOT NULL,
  day integer NOT NULL,
  prec integer NOT NULL,
  tmax integer NOT NULL,
  tmin integer NOT NULL,
  wind integer NOT NULL,
  CONSTRAINT city_data_pkey PRIMARY KEY (id ),
  CONSTRAINT city_data_city_id_fkey FOREIGN KEY (city_id)
      REFERENCES city_place (id) MATCH SIMPLE
      ON UPDATE NO ACTION ON DELETE NO ACTION DEFERRABLE INITIALLY DEFERRED
)
```

Figure 13: Table of City and Data

Contains the data of temperature, wind, precipitation that matches with the cities.

- **City Place**

```
CREATE TABLE city_place
(
  id serial NOT NULL,
  city character varying(50) NOT NULL,
  slug character varying(50) NOT NULL,
  longitude numeric(5,2) NOT NULL,
  latitude numeric(5,2) NOT NULL,
  image character varying(100),
  CONSTRAINT city_place_pkey PRIMARY KEY (id ),
  CONSTRAINT city_place_slug_key UNIQUE (slug )
)
```

Figure 14: Table of City and Location

Contains the name, the longitude and latitude of each city.

- **City Searchplace**

```
CREATE TABLE city_searchplace
(
  id serial NOT NULL,
  city_id integer NOT NULL,
  "fromDate" date NOT NULL,
  "toDate" date NOT NULL,
  age integer NOT NULL,
  gender character varying(1) NOT NULL,
  avg_tp integer NOT NULL,
  created_on timestamp with time zone NOT NULL,
  CONSTRAINT city_searchplace_pkey PRIMARY KEY (id ),
  CONSTRAINT city_searchplace_city_id_fkey FOREIGN KEY (city_id)
      REFERENCES city_place (id) MATCH SIMPLE
      ON UPDATE NO ACTION ON DELETE NO ACTION DEFERRABLE INITIALLY DEFERRED
)
```

Figure 15: Table of the Cities to Search

Contains the data of temperature, wind, precipitation that matches with the cities.

- **City User-Profile:**

  This table contains the information from the user, extracted from Facebook. Also, this is the table that we are filling to do the data mining, since we are saving City, fromDate, toDate (range of date), age, gender, avg tmp(the average of the temperature for the range), created on the dateTime when the user did the search. Based on these parameters, next time the user does a search, we are going to suggest cities, based on the age, the gender, and the average of temperature.

```
CREATE TABLE city_userprofile
(
  id serial NOT NULL,
  about_me text,
  facebook_id bigint,
  access_token text,
  facebook_name character varying(255),
  facebook_profile_url text,
  website_url text,
  blog_url text,
  date_of_birth date,
  gender character varying(1),
  raw_data text,
  facebook_open_graph boolean,
  new_token_required boolean NOT NULL,
  image character varying(255),
  user_id integer NOT NULL,
  CONSTRAINT city_userprofile_pkey PRIMARY KEY (id ),
  CONSTRAINT user_id_refs_id_66f9def2 FOREIGN KEY (user_id)
      REFERENCES django_facebook_facebookcustomuser (id) MATCH SIMPLE
      ON UPDATE NO ACTION ON DELETE NO ACTION DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT city_userprofile_facebook_id_key UNIQUE (facebook_id ),
  CONSTRAINT city_userprofile_user_id_key UNIQUE (user_id )
)
```

Figure 16: Table of the User Cities Searched

3.2 Interface

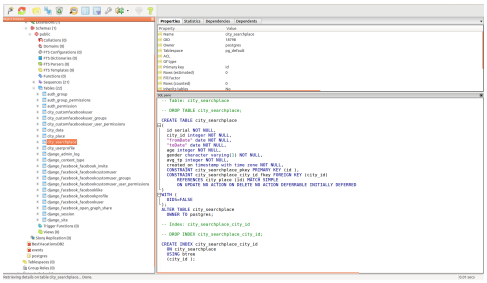

Figure 17: Schema of the Interface.

- **Appearance**
  An example of our interface structure development is shown in the image 18. The features that we display are sign in, search, graph, and suggested cities. The design structure of each of these is covered below.

- **Structure**

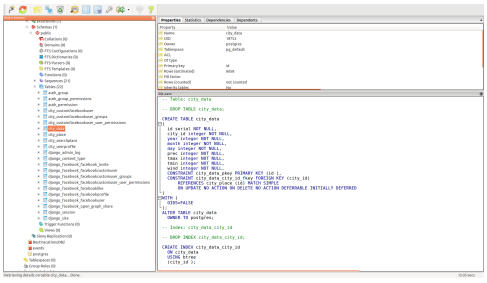

Figure 18: Schema of the Interface

The structure of our interface is modeled by Django through its queries techniques. We used these techniques to populate the user table, to query the other tables in our database, and to perform some data mining techniques. Django provides for an intuitive way of representing the database in python. It uses a model class for database tables where an instance of a class represents a record of each table, as shown in the image below.

- **Creating Objects**

  The class mentioned above represents a table of the database named Blog in python. To create an object, we would need to instantiate this class using keywords to model class, and then save the changes. The class needs to be imported by using the python path to the class. Assuming models live in a file mysite / blog / models.py, The code in figure 17 adds a record to Blog().

- **Updating Objects**

  This is an intuitive task where we perform the updating in the class. Next, we ask the database to save the changes. The example in figure 18 changes the name of the instance of the class b5.

- **Retrieving Objects**

  This is done using the Query set on the manager. Queryset in Django is equivalent to a select statement. In the example below, the object represents the default manager and Blog() represents the query set with a few filters that filter out the results. Next, a number of filters can be used on the resultant query. Keywords like filter and exclude are used for that. The keyword $'all'$ is used if all the results are required. In case we require only a single output the keyword get is used. We could also limit the results such that we get a range of results from the table.

- **Sign-In**

  The table Person is very necessary for the registration process, this can be explained as follows. To register the user we need the user's name and password. This data is only used for registration and user identification process. All the data gathered is saved in a table named Person, and the reason why is important is because this table allows us to identify users and predict users' behavior according to their search history. The registration process works in the following manner. Actually, the only way for users to sign in is through their Facebook accounts. We create a temporary table of this user to check if the user is not in our system. If the user is already in our database we just update the user's table. Otherwise, we add the temporary table as a new user and update our homepage.

The architecture of our system can be improved by understanding the complexity of our system. We recognized common paradigms which allow high level relationships among systems to be understood. In this way, we will be able to build a new system as variations on our current systems.

4. IMPLEMENTATION

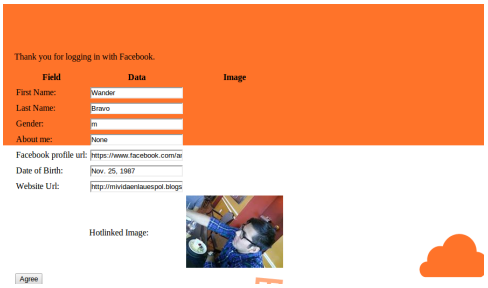

Figure 19: Sign-In - Interface Development

The Implementation section describes how system applications will be installed, deployed and transitioned into an operational system for the purpose of our project. In this section we will give an overview of the system, describe major tasks involved in the implementation process, and the overall resources needed to support the implementation effort such as hardware, software, facilities, techniques, among others. Furthermore, we will present users with the documentation and tools necessary to make an informed decision on how to deploy our system for short-term vacationing. In addition, multiple features have been added and we will explain how to utilize them in our system during its implementation process. We will demonstrate how these applications, based on our interface and database sections, can be installed and configured in any environment by deploying the enabled operations of our system that have been completely developed and tested for our project.

4.1 Interface

1. Registration

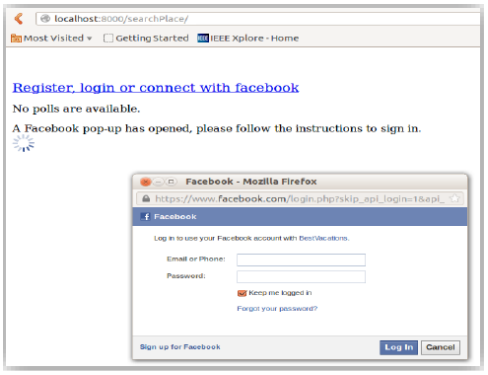

Figure 20: Registration - Interface Development.

In order for users to perform any kind of task, users need to sign into our system through their Facebook account. Once users are registered in our system, we get access to their basic information such as name, gender, age, among others. Django easily connects python to Facebook in order to process the necessary information for a given query.

2. Search

Figure 21: City Search- Interface Development

In this feature we are searching into our database through Django for a particular city. This is done by entering the city name in Django, which queries into the database class.

3. Graph

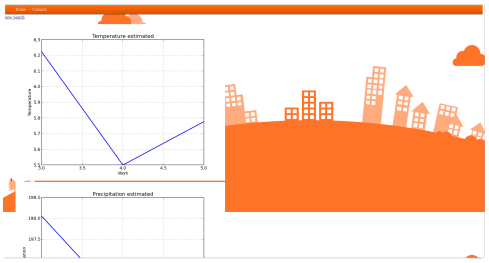

Figure 22: Weather Graphical Approximation - Interface Development.

The graph is generated through the function that we have built into Django. This takes take the regression of the data of each day within a given range. Then, we plot the graph with the resulting data using python. This data is an approximation of the weather that the user has selected for vacationing purpose, an example of this function is in figure 23.

4. Suggested List

Every time the user searches for a city, this search is being saved by Django in a separate table that belongs to the user. Also, it stores the average temperature of the queried destination. Next, we query into our database using the average temperature to suggest the

```
t1 = data.objects.filter(city_id = city, day = x , month = month1).aggregate(Avg('tmax'))
```

Figure 23: Weather Approximation Function

user with a list of appropriate destinations.

These components were planned, analyzed, and designed into our system through documentation such as data flow diagrams and others that will guide users to use our system if any issue arises. We have tested our system several times to make sure that our system works and performs the duties it is intended to perform. The main goal of our testing process was to detect any problem or error as early as possible in the implementation process. Therefore, it was critical to make sure that each application was implemented in accordance with our design and architecture plan to ensure that our system is ready for implementation. In this way, we were able to determine issues, occurred during the implementation process, which affected our system. In summary, we have established an overall development with the best practices to achieve the goal of our system implementation.

5. LESSONS LEARNED

5.1 Techniques

We learned numerous database and data mining techniques while working on this project. We learned a number of ways of formatting our data so that we can easily use it in our application. The data downloaded from the website was raw and faulty, we added dates corresponding to each row to make it more meaningful. We also related each longitude and latitude pair to a city. And we learned how to convert text files into mysql database so as to be able to use in our project.

We learned how to query on a huge database. As a result of querying for the weather conditions of a place, we get a number of results by using regression techniques on our database. This provides a single value for each corresponding date that the user is going to be on vacation.

We learned a few data mining techniques too. We are able to suggest a few vacation destinations for our users, making our application more interactive. The suggestions were based on the age, gender and the last search made by the user. Using these variables we perform data mining, as mentioned above, to search in our database for more destinations that they user may like.

6. CURRENT STATUS

The current status of our project is being explained through the three main sections that highly influenced the development of our project.

First, we gathered the weather data of a few cities of U.S.A. This data consists of temperature, precipitation and wind speed corresponding to each city from 1980 to 2013. Each city is related to an id, latitude, and longitude.

Second, we developed an interface to provide the user with a GUI to query efficiently on the database. The interface asks the users to sign-in to proceed searching for destinations to vacation in a given range of dates. The sign-in is done through Facebook, which allows the user to avoid the whole process of signing in. Once signed-in, the user will be given a list of suggested cities to vacation. The users can choose a place from this list, or search for their city of preference to vacation, and the vacation time duration. Then, the application searches for the corresponding weather data of the place and graphically displays the approximate temperature, precipitation, and wind speed of the place.

Lastly, the suggested cities that we display, before and after the user has searched, is done through a data mining technique known as association rule discovery. In fact, this list of suggested cities provides our project with an interactive application.

7. FUTURE WORK

- **Big Data**
  More research is needed to improve the ability to blend different datasets, and data sources into integrated systems. As new types, data could be assimilated into different models. Also, it will be important to understand the error characteristics of our current data and our model to come up with modifications that may improve our system. Data assimilation for climate purposes is still in an early stage of development and requires continued research support. As these developments occur, reprocessing of data to take advantage of the new knowledge will be vital if we want to sustain long-term records in the future.

- **Data Mining**
  If our weather observations were well-documented, and it contained good metadata about the systems and networks used to make them, it would become more valuable with time. The creation of weather-quality data records is a fundamental objective for our observing system for weather. International standards and procedures for the storage and exchange of metadata need to be developed and implemented for many weather observing system components, including those of the operational satellite community. For this, it will be essential that all such data be properly archived and managed with the full expectation that they will be reused many times over in the future, often as a part of reprocessing or reanalysis activities. Future modifications may require that data be migrated to new media as technology changes, be accessible to users, and be made available with minimal incremental costs. For this, we will need systems for quality control and feedback, proper data archival and access, sustained data preservation, rescue and digitization, and climate data management systems.

8. CONCLUSION

This project, "Best Time To Vacation," was prepared in response to avoid unexpected bad weather during a vacation trip. The full implementation of time project, over the coming years, will ensure users to have the observational information needed to understand, predict, and manage their response to weather and weather change, over the 21st century and beyond, for vacationing purposes. It will address the commitments of the users to certain places, and support their needs for weather observations in fulfillment of the objectives of their vacation.

The database sustains observations of the essential climate variables that are used to make significant progress in the generation of weather analysis and derived information for our project. These observations give an enhanced support to our research, modeling, and analysis for users' s vacationing events. In fact, these observational records improve weather predictions for short-term vacations. These observations must be sustained into the future to evaluate how the weather is changing so that informed decisions can be made by any user on prevention, mitigation, and strategies when vacationing. It is crucial for users to support the further research needed to refine understanding of the weather approximated and its changes. This will allows us to initialize predictions on time scales out to decades ahead, and to develop the models used to make these predictions and short-term scenario-based projections.

9. REFERENCES

[1] NCDC. Babel, Whether Datasets. *cdo-web*, 12(2):291–301, June 1991. http://www.ncdc.noaa.gov/cdo-web/

[2] M. Clark. Post congress tristesse. In *TeX90 Conference Proceedings*, pages 84–89. TeX Users Group, March 1991.

[3] M. Herlihy. A methodology for implementing highly concurrent data objects. *ACM Trans. Program. Lang. Syst.*, 15(5):745–770, November 1993.